PayPal AI Hackathon

Team Briefing Document

FinPal — Financial Agent Buddy

Research, Analysis, Feasibility, Stack, Costs & Recommendations

Prepared for the team | 14 October 2026

Deadline: 12 November 2026, 9:00 PM Lagos time

1. Hackathon Overview

The PayPal AI Hackathon (paypalaihackathon.devpost.com) is a large-scale online event focused on combining PayPal tools with AI, with a strong emphasis on **agentic commerce**. Participants must meaningfully combine the PayPal sandbox with AI. There are no prescribed tracks, but special prizes exist for agentic commerce, best demo, most impactful, and sponsor-tool usage.

Item	Detail
Scale	~13,500+ expected participants
Submission Deadline	12 November 2026, 9:00 PM Lagos time
Project Types Allowed	New projects and existing/upgraded projects
Key Prize Categories	Grand Prize, Best Use of Agentic Commerce, Best Use of PayPal + AI, Best Demo Delivery, Most Impactful, Most Creative, Sponsor prizes (AG Grid, APIMatic, Bryntum, etc.)
Judging Criteria	Technological Implementation, Design, Potential Impact, Innovation/Idea, Presentation
Core Requirement	Meaningful PayPal sandbox + AI integration (not just a checkout button)

2. Past Winners Research

Research focused on the most recent and relevant PayPal events (Dev Days 2025 and PYUSD Portal 2024). Older Battle Hack events (2013–2015) have sparse public documentation.

Dev Days 2025 Top Projects

Place	Project	Key Characteristics
1st	CrawlerPal	Agentic payment flow using L402 protocol + PayPal. Agents pay for content. Clear end-to-end loop. FastAPI + Python. Small team.
2nd	Codebusters	Practical modernization — upgraded Shopware from old JS SDK (v5) to new v6. Measurable performance improvement.
3rd	Rosen	Upgrade of an existing social trading product with PYUSD integration.

Key takeaway: The top project was an agent that actually transacted. Practical upgrades of existing products also placed well. Working prototypes with clear demos were essential.

3. Winning Patterns

Pattern	Why It Wins	Relevance to FinPal
Agent that actually transacts	Directly matches agentic commerce focus	Strong match if agent makes real sandbox payments
Meaningful PayPal use	Not just a button; uses Orders, webhooks, authorise-capture, etc.	Yes — constraints + payment execution
Working end-to-end prototype	Judges must see the full flow	Critical success factor
Clear specific problem	Easy for judges to understand impact	Spending control + safety is clear

Pattern	Why It Wins	Relevance to FinPal
Focused scope	Small teams with tight scope outperform broad incomplete projects	Must cut full vision to focused slice
Strong demo narrative	Presentation is a scored criterion + separate prize	Demo video quality matters
Safety / quality layer	Differentiates from pure permission systems	Purchase quality score addresses biggest gap

4. Agentic Commerce Safety Gaps

Analysis of three existing project concepts (Proof of Intent, Passport, Handover) showed they all address the question “Should this agent be allowed to spend?” They leave open identity and everything that happens after authorization.

The Three Existing Concepts

Project	Question It Answers	What It Guards
Proof of Intent	Did the human really ask for this?	Permission (made unfakeable)
Passport	Is this a well-behaved agent?	Reputation across shops
Handover	Did these two agents agree?	Signed deal between agents

Identified Gaps (White Space)

- Identity foundation — authenticating the agent and tying it to a real accountable person (assumed solved by the three above).
- Gap 1 (Biggest): Did the agent make a good decision, not just an allowed one? (purchase quality)
- Gap 2: Stopping or clawing back a running agent (real-time revocation).
- Gap 3: Who is liable when it goes wrong (dispute and redress path).
- Gap 4: Is the shop safe for the agent? (merchant trust).
- Gap 5: Aggregate control across a person’s many agents.

Strongest recommended direction from the analysis: Gap 1 + Gap 3 combined — a referee that checks whether an agent’s purchase was actually a good outcome and routes redress when it was not.

5. FinPal — Financial Agent Buddy

FinPal is a personal/business AI agent that turns a passive PayPal account into an active assistant. The full vision in the source notes is broad; the hackathon submission should be a focused slice.

Full Vision Features (from source notes)

- Predictive Spending Controls — calendar + PayPal history warnings before high-expense periods.
- Schedule-Linked Subscriptions — pause gym/streaming when travel is detected.
- Conversational Business Intelligence — plain-English queries on customer/vendor metrics.

- Autonomous Spending with Strict Constraints — time windows + granular caps using PayPal reference tokens.
- Omnichannel Notifications — WhatsApp one-tap approval.
- Anti-Fraud Duress Detection — behavioral analytics, IP-to-phone location matching, biometric step-up.
- Email-to-Crypto Address Masking.

Recommended Focused Scope for Hackathon

Calendar trigger (Google only) → Constraint check (time windows + amount caps) → Purchase quality score → PayPal authorise-and-capture → Approval or block → Optional revocation → Activity log.

6. Achievability Assessment

Version	Achievable with Team of 4?	Competitive Potential
Full FinPal vision (all features)	No	—
Focused slice (core loop)	Yes	High
Focused slice + 1–2 extras (quality score, revocation, AG Grid)	Yes	High

A team of 4 can deliver a polished focused version. Trying to implement the entire blueprint risks an incomplete demo, which historically does not place well.

7. Prize Odds (Focused + Polished Version)

Outcome	Approximate Odds	Conditions
Special prize (Agentic Commerce, PayPal+AI, Demo, etc.)	Medium-High	Clean end-to-end demo + safety angle
Top 10–20 overall	Medium	Working prototype + clear narrative
Grand Prize	Low–Medium	Possible but competitive field
No prize	Medium	If demo is weak or many stronger entries

The project matches patterns that have won past PayPal events (especially CrawlerPal's agentic transaction focus). It is more practical and judge-friendly than many protocol-heavy ideas. Main variable is execution quality and demo clarity.

8. Achievable Features That Improve Odds

Feature	Why It Helps	Effort
Purchase Quality Score	Addresses biggest gap; makes agent feel intelligent	Medium
Instant Revocation / Kill Switch	Uses authorise-and-capture; strong technical + agentic signal	Medium
Simple Merchant Trust Check	Safety differentiator	Low–Medium
Clear Transaction Audit Trail	Improves Design and Presentation scores	Low

Feature	Why It Helps	Effort
AG Grid Activity Dashboard	Sponsor prize + better UI	Low–Medium
One-tap Email/WhatsApp Approval	Stronger demo	Medium

Highest-leverage additions: Purchase Quality Score + clear audit trail. Best package: Quality Score + Revocation + AG Grid dashboard.

9. Recommended Tech Stack

Layer	Recommended Choice	Notes
Frontend	Next.js (App Router) + React + Tailwind + shadcn/ui	Fast, modern, good for dashboards
Data Display	AG Grid (optional)	Sponsor prize potential
Backend	Next.js API routes or FastAPI	Next.js keeps monorepo simple
Database	Supabase (Postgres)	Free tier + auth
AI / Scoring	Groq (Llama) or OpenAI / Claude	Groq free tier is sufficient
PayPal	Sandbox + Orders API v2 + authorise-and-capture	Enables revocation
Calendar	Google Calendar API	One source is enough
Notifications	Resend (email) or Twilio	Simple approval messages
Deployment	Vercel	Free tier, easy demos
Auth	Supabase Auth or NextAuth	Quick setup

No smart contracts are required. The project is a standard web application. PayPal developer terms are handled by creating a sandbox account.

10. Costs and Testing

Total typical cost for the team during the hackathon: **$0–50** (most teams stay under $20 or at $0).

Item	Cost	Notes
PayPal Sandbox	Free	No real money
Google Calendar API	Free	Generous quota
Groq / LLM API	$0–20	Free tier usually enough
Supabase	Free	Free tier sufficient
Vercel / Render	Free	Free tier works
Resend (email)	Free	Free tier
Total	$0–50	Sandbox + free tiers

Why Testing Does Not Require Real Money

PayPal Sandbox is a fully functional environment that behaves exactly like live PayPal, except money is simulated. You can create sandbox buyer and seller accounts, authorise payments, capture them, void them, receive webhooks, and see transaction history — all with fake money. From the app's perspective the API calls and responses are identical to production. You stay in sandbox for the entire hackathon.

11. Requirements to Complete a Focused Version

Accounts & Access

- PayPal Developer account + Sandbox credentials
- Google Cloud project with Calendar API enabled
- Groq (or OpenAI/Claude) API key
- Supabase project
- Vercel account
- (Optional) Resend or Twilio

Core Components to Build

- User authentication
- Google Calendar connection (OAuth + read events)
- PayPal sandbox connection
- Constraints engine (time windows + spending caps)
- Purchase quality scorer
- Decision flow (trigger → checks → approve/block)
- Payment execution (authorise-and-capture)
- Revocation / kill switch
- Activity log
- Basic UI (dashboard, settings, history)
- Notifications (optional)

Suggested Team Split (4 people)

Person	Primary Ownership
1	PayPal + Constraints backend (rules, payment, revocation)
2	Calendar + Trigger logic
3	Frontend + Dashboard / UI
4	AI scoring + Notifications + Documentation + Demo video

What "Done" Looks Like

A judge can log in, connect sandbox PayPal and Google Calendar, set simple rules, trigger or see a calendar event, watch the agent run constraint + quality checks, see the payment blocked or completed, revoke an in-flight payment, and view the decision history. Plus a clear demo video and solid README.

12. Summary & Recommended Next Steps

FinPal is well-aligned with what wins PayPal AI hackathons when kept focused. The full vision is a strong long-term product idea, but the hackathon submission should be a tight, working agentic loop that demonstrates constrained, safety-checked payments.

Immediate Next Steps

1. Agree on the exact focused scope (core loop + which 1–2 extras).
2. Create shared accounts (PayPal Sandbox, Supabase, Groq, Google Cloud, Vercel).
3. Set up the repository and basic Next.js + Supabase skeleton.
4. Implement the PayPal authorise-and-capture flow first (so you always have a working payment demo).
5. Add constraints engine and calendar trigger.
6. Add purchase quality score and audit trail.
7. Polish UI, write documentation, and record the demo video in the final week.

This document consolidates all research, analysis, and recommendations from the planning discussions. Use it as the shared reference for scope decisions and execution.